

Análisis de un Caso Real de Arquitectura de Microservicios: Uber

Ivan Andrés Villamarín Cuenca

Aplicaciones Distribuidas

FORM-IC-01 — Investigación Individual — Arquitecturas Distribuidas en la Industria

<https://github.com/ivillamarinc/aplicaciones-distribuidas>

Resumen—Este artículo analiza la migración de Uber desde una arquitectura monolítica hacia una arquitectura de microservicios, examinando las motivaciones, beneficios y desafíos que implicó dicha transición. Asimismo, se evalúa la aplicabilidad de este enfoque arquitectónico en el contexto de un sistema de control de laboratorios informáticos, identificando cómo los principios de escalabilidad, independencia y mantenibilidad de los microservicios pueden adaptarse a entornos académicos de menor escala.

Palabras clave—microservicios, arquitectura distribuida, escalabilidad, monolito, Uber, laboratorios informáticos.

I. Introducción

Uber es una de las empresas más conocidas por utilizar una arquitectura de microservicios en sus sistemas. Cuando la empresa comenzó a operar, empleaba una aplicación monolítica donde todas las funciones estaban estrechamente acopladas entre sí. Este modelo funcionó bien al inicio; sin embargo, a medida que aumentó la cantidad de usuarios, conductores y ciudades donde prestaba sus servicios, comenzaron a surgir dificultades para mantener y ampliar el sistema.

Para solucionar estos problemas, Uber decidió migrar hacia una arquitectura basada en microservicios, una estrategia ampliamente utilizada por organizaciones que necesitan sistemas más flexibles y escalables [1].

II. Fundamentos de los Microservicios

Los microservicios consisten en dividir una aplicación en varios servicios pequeños e independientes. Cada uno de estos servicios se encarga de una tarea específica, como gestionar los viajes, procesar pagos, administrar la información de los usuarios o enviar notificaciones. De esta manera, cada parte del sistema puede desarrollarse, actualizarse y mantenerse sin afectar directamente a las demás [2].

Este paradigma contrasta con la arquitectura monolítica tradicional, en la que todos los componentes se despliegan como una única unidad. La independencia que ofrecen los microservicios permite que los equipos de desarrollo trabajen en paralelo sobre distintas funcionalidades, acelerando los ciclos de entrega y reduciendo el riesgo de introducir errores en partes no relacionadas del sistema.

III. Motivaciones de la Migración en Uber

La principal razón por la que Uber eligió esta arquitectura fue la necesidad de crecer sin afectar el funcionamiento de toda la plataforma. En el sistema anterior, cualquier cambio o actualización requería modificar una gran parte de la aplicación, lo que aumentaba el riesgo de errores y ralentizaba el trabajo de los desarrolladores. Con los microservicios, cada equipo puede trabajar en una función específica sin interferir con las demás, lo que facilita el desarrollo y el mantenimiento del sistema [1].

La escalabilidad horizontal también fue un factor determinante: al segmentar las responsabilidades en servicios autónomos, es posible asignar recursos de cómputo de manera granular y proporcional a la demanda real de cada componente.

IV. Beneficios Obtenidos

Entre los beneficios obtenidos por Uber destacan los siguientes:

Escalabilidad independiente: Si la función encargada de gestionar viajes recibe un volumen elevado de solicitudes, únicamente ese servicio requiere recursos adicionales, sin impactar a los demás componentes.

Tolerancia a fallos: Si ocurre un problema en una parte del sistema, las demás pueden continuar funcionando, lo que mejora la disponibilidad y estabilidad de la plataforma.

Velocidad de despliegue: La independencia entre servicios permite implementar nuevas características con mayor rapidez y menor coordinación entre equipos.

Flexibilidad tecnológica: Cada microservicio puede implementarse con el lenguaje o tecnología más adecuada para su función específica.

Estas ventajas coinciden con lo reportado en estudios recientes sobre arquitecturas de microservicios, donde se destacan la flexibilidad, la escalabilidad y la independencia de los servicios [2].

V. Desafíos y Retos

Sin embargo, la adopción de microservicios también trajo consigo desafíos significativos. Al aumentar la cantidad de servicios independientes, se volvió más difícil controlar la comunicación entre ellos y supervisar que todos funcionaran correctamente. Asimismo, coordinar el trabajo de numerosos equipos y mantener organizada toda la infraestructura requirió nuevas herramientas y estrategias de gestión [3].

Diversas investigaciones señalan que la complejidad operativa, el monitoreo distribuido y la gestión de datos en entornos heterogéneos son algunos de los principales retos de esta arquitectura [3, 4]. En particular, la descomposición de aplicaciones monolíticas hacia microservicios exige un análisis cuidadoso de las fronteras de dominio y de las dependencias

existentes [4].

ring, vol. 49, no. 8, pp. 4213–4242, ago. 2023, doi: 10.1109/TSE.2023.3287297.

VI. Aplicación al Sistema de Control de Laboratorios Informáticos

La arquitectura de microservicios es aplicable al diseño de un sistema de control de laboratorios informáticos. Este enfoque significaría dividir el sistema en servicios independientes, donde cada uno se encargue de una función específica:

Gestión de usuarios: Registro, autenticación y administración de roles (estudiantes, docentes, administradores).

Reservas: Control de disponibilidad y asignación de laboratorios y equipos.

Control de equipos: Monitoreo del estado y condición del hardware disponible.

Reportes: Generación de informes de uso, incidencias y estadísticas de ocupación.

De esta forma, si se necesita modificar o mejorar una funcionalidad, como el módulo de reservas, no sería necesario actualizar todo el sistema. Además, cada servicio podría escalar de manera independiente según la demanda, facilitando el mantenimiento, la incorporación de nuevas características y la resiliencia ante fallos parciales.

VII. Conclusión

El caso de Uber demuestra que la arquitectura de microservicios puede ser una solución efectiva para organizaciones que requieren crecer constantemente y ofrecer servicios confiables a grandes volúmenes de usuarios. Aunque este enfoque aporta ventajas importantes en términos de escalabilidad y flexibilidad, también exige una buena organización, herramientas de observabilidad adecuadas y un diseño cuidadoso de las fronteras entre servicios para evitar que la complejidad operativa se vuelva inmanejable.

En el contexto de un sistema de control de laboratorios informáticos, la adopción de microservicios representa una oportunidad para construir una plataforma modular, extensible y robusta, siempre que se planifique correctamente la descomposición del dominio y la comunicación entre servicios.

Referencias

- [1] H. M. Ayas, P. Leitner, R. Hebig, X. Peng, B. Hamdy y M. Ayas, “An empirical study of the systemic and technical migration towards microservices,” *Empirical Software Engineering*, vol. 28, no. 4, pp. 85–, mayo 2023, doi: 10.1007/S10664-023-10308-9.
- [2] V. Velepucha y P. Flores, “A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,” *IEEE Access*, vol. 11, pp. 88339–88358, 2023, doi: 10.1109/ACCESS.2023.3305687.
- [3] M. Söylemez, B. Tekinerdogan y A. K. Tarhan, “Challenges and Solution Directions of Microservice Architectures: A Systematic Literature Review,” *Applied Sciences*, vol. 12, no. 11, p. 5507, mayo 2022, doi: 10.3390/APP12115507.
- [4] Y. Abgaz *et al.*, “Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review,” *IEEE Transactions on Software Engineer-*